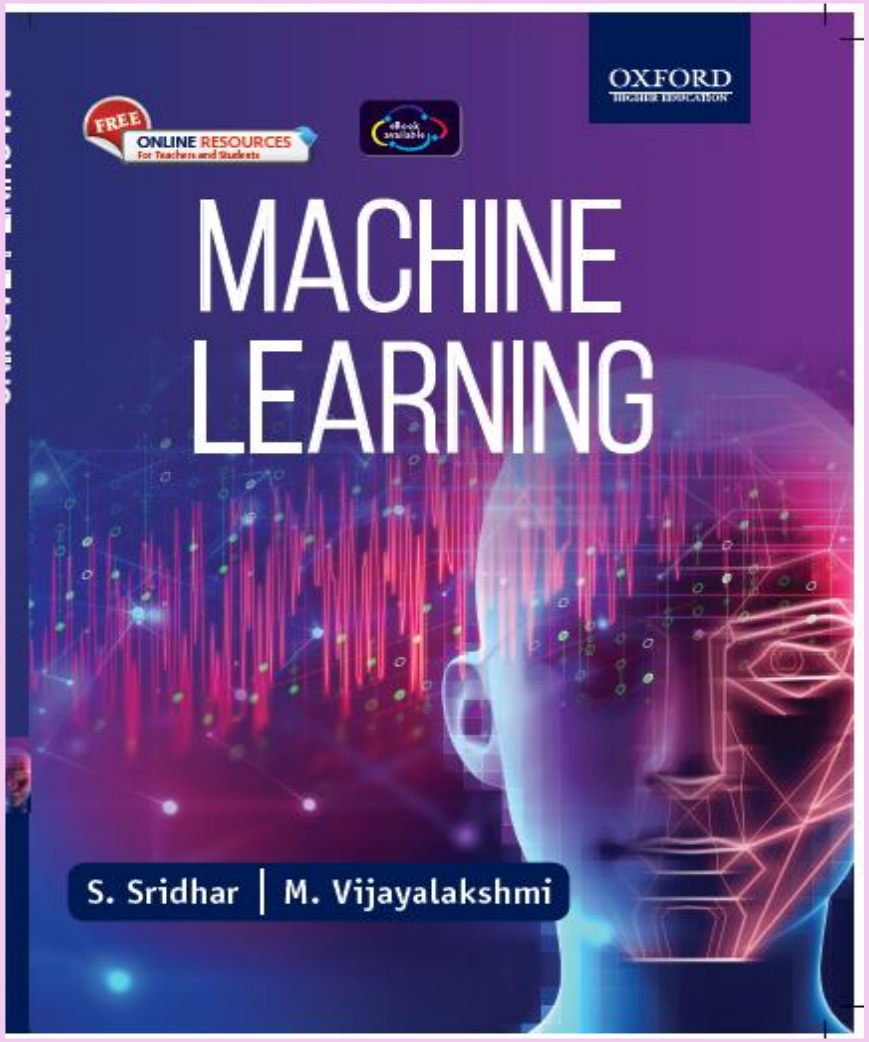


OXFORD
UNIVERSITY PRESS

Machine Learning

S. Sridhar and M. Vijayalakshmi

© Oxford University Press 2021. All rights reserved

Chapter 14

Reinforcement Learning

© Oxford University Press 2021. All rights reserved

Reinforcement Learning

- REINFORCEMENT LEARNING IS LEARNING FROM ENVIRONMENT USING REWARDS

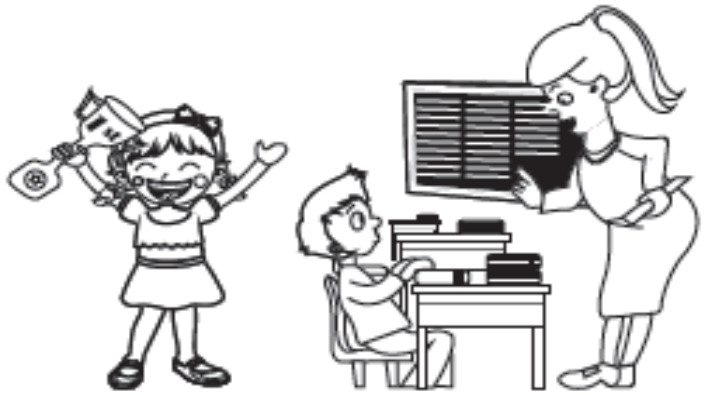


Figure 14.1: Examples of Positive and Negative Rewards

© Oxford University Press 2021. All rights reserved

Scope of Reinforcement Learning (RL)

- USEFUL FOR SOLVING PROBLEMS WITH MORE UNCERTAINTY LIKE GRID GAMES

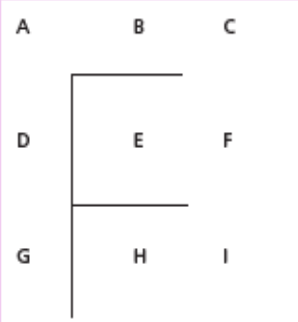


Figure 14.2: A Grid Game

© Oxford University Press 2021. All rights reserved

Characteristic of RL

1. Sequential decision making – Consider the Figure 14.3. It can be seen the path from start to goal is not done in one step. It is a sequence of decisions that leads to the goal. One wrong move may result in a failure. This is the main characteristic of reinforcement learning.
2. Delayed feedback – Often, rewards are not immediate. One must spend many moves to get final success or failure. Feedback in terms of reward is often delayed.
3. The agent actions are interdependent as any action affects the subsequent actions. For example, one wrong move of an agent may lead to failure.
4. Time related – All actions are associated with time stamps inherently as all actions are ordered as per the timeline inherently.

© Oxford University Press 2021. All rights reserved

RL Challenges

1. Reward design is a big challenge as in many games, as determining the rewards and its value is a challenge.
2. Absence of a model is a challenge – Games like chess have fixed board and rules. But, many games do not have any fixed environment or rules. There is no underlying model as well. So, simulation must be done to gather experience.
3. Partial observability of states – Many states are fully observable. Imagine a scenario in a weather forecasting where the uncertainty or partial observability exists as complete information about the state is simply not available.
4. Time consuming operations – More state spaces and possible actions may complicate the scenarios, resulting in more time consumption.
5. Complexity – Many games like GO are complicated with much larger board configuration and many possibilities of actions. So, labelled data is simply not available. This adds more complexity to the design of reinforcement algorithms.

© Oxford University Press 2021. All rights reserved

RL Applications

- Industrial automation
- Resource management applications to allocate resource
- Traffic light controller to reduce congestion of traffic
- Personalized recommendation systems like news
- Bidding for advertisement

- Customized applications
- Driverless cars
- Along with deep learning games like Chess and GO
- Deep mind applications like to generate programs and images

© Oxford University Press 2021. All rights reserved

RL Vs Supervised Learning

Table 14.1: Differences between Reinforcement Learning and Supervised Learning

Reinforcement Learning	Supervised Learning
No supervisor and labelled dataset initially	Presence of supervisor and labelled data
Decisions are dependent and are made sequentially	Decisions are independent of each other and based on input given in the training phase
Feedback is not instantaneous and delayed by time	Usually, the feedback is instantaneous once the model is created
Agent action affects the next input data	Dependent of initial input or the input given at start
No target values, only goal oriented	Target class is predefined by the problem
Example: Chess, GO, Atari Games	Example: Classifiers

© Oxford University Press 2021. All rights reserved

RL Vs Unsupervised Learning

Table 14.2: Differences between Reinforcement Learning and Unsupervised Learning

Reinforcement Learning	Unsupervised Learning
Mapping from input to output is present	Not present
Get constant feedback from environment	No feedback from environment

© Oxford University Press 2021. All rights reserved

Components of RL

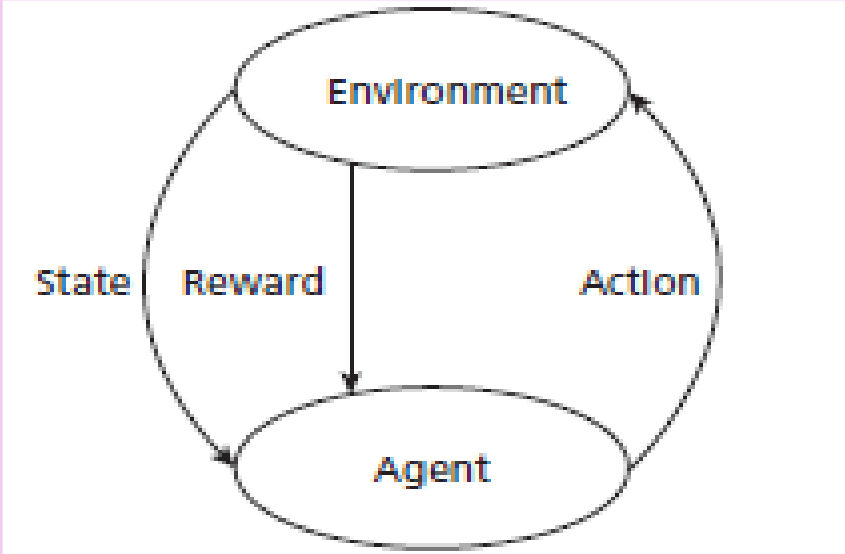


Figure 14.4: Basic Components of RL

© Oxford University Press 2021. All rights reserved

Policies

Policies

The role of algorithms is called a policy in reinforced learning and policy maps states to actions. In short, a policy takes the input states, processes it and suggests an action. Policies characterize the agent behaviour function. There are two types of policies – deterministic and stochastic.

The deterministic policy function is given as:

$$a = \pi(s) \tag{14.1}$$

A deterministic policy states that a given state always returns the same action, i.e., premapped.

The stochastic policy function is given as:

$$\pi(a \mid s) = Pr[a_t = a \mid s_t = s] \tag{14.2}$$

It is the probability of taking an action '*a*' while in state '*s*'. The aim of the agent is to learn policy function so that the best actions are selected.

© Oxford University Press 2021. All rights reserved

Rewards

Immediate Reward The environment immediately delivers a reward after the execution of action and the total reward is associated with the episode. This is the sum of all rewards that come from the first move to the last mode. The total reward is also called as return or goal.

It can be observed that every game has states. It then selects an action and transits to another state. As a result of the transition, it collects the rewards too. This continues till the terminal state. The return is given as:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \ldots + r_T \quad (14.3)$$

Here, G_t is the return and r_T is the terminal or final state.

© Oxford University Press 2021. All rights reserved

Rewards

The total future reward is the sum of the current move reward and all other moves' rewards till the end of the episode. For example, if there are 9 moves in an episode, the total future reward of the third move will be the sum of reward of that move and rewards of move 4 to move 9 present in that episode.

More often, the iteration can be without terminal state too. In other words, it is a continuous task. To compute future rewards or utility, one can introduce a factor γ called discount factor:

$$G_t = \gamma_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \ldots$$

$$= \sum_{j=0}^{\infty} \gamma^j r_{t+j+1} \quad (14.4)$$

© Oxford University Press 2021. All rights reserved

Markov Decision Process

Markov Decision Process (MDP) is an extension of Markov Chain. What is a Markov chain? A Markov chain is a stochastic process satisfying Markov property. It is a sequence of random variables $X_0, X_1, \ldots, X_n$ satisfying conditional probability.

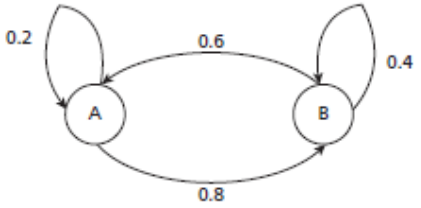


Figure 14.6: Markov Chain

© Oxford University Press 2021. All rights reserved

Example

Example 14.2: Assume the universities' hold on a town is initially 60% and 40%. So, what would be the prediction after one month, that is, the hold of university after an year. After two years, will the hold remain same or change? Assume the rest of the probability transition information from Figure 14.6.

Solution: The starting distribution is $u_0 = (60\%, 40\%) = (0.6, 0.4)$. The Markov chain is shown in Figure 14.6 and the transition matrix is given as:

$$P = \begin{pmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{pmatrix}$$

The prediction after a year as in Eq. (14.5), $u_1 = u_0 \times P$, will be:

$$= (0.6 \quad 0.4)\begin{pmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{pmatrix}$$

$$= \begin{pmatrix} 0.28 \\ 0.72 \end{pmatrix}$$

The hold of the universities will be 28% and 72%. This is obvious because 80% of the students shift from the university A to B.

The second-year prediction $u_2 = u_1 \times P$ would be:

$$= (0.28 \quad 0.72)\begin{pmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{pmatrix}$$

$$= \begin{pmatrix} 0.344 \\ 0.656 \end{pmatrix}$$

It can be observed that the hold on the student by the universities changes.

© Oxford University Press 2021. All rights reserved

Markov Decision Process

1. A set of states
2. A set of actions
3. Reward Function R
4. Policy π
5. Value V

© Oxford University Press 2021. All rights reserved

Multi-Arm Bandit Problem

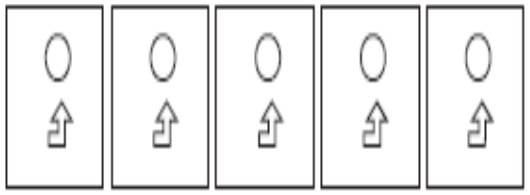


Figure 14.7: Arm Slot Machines

$$Q_k(a) = \frac{r_1 + r_2 + \cdots + r_k}{k} \tag{14.9}$$

This function is called action-value function or Q function. This indicates the value of taking a particular action 'a'.

$$Q_* = \arg\max_a Q_k(a_i) \tag{14.10}$$

The best action (highest Q_* value) is the action that returns the highest average return and is the indicator of the action quality.

© Oxford University Press 2021. All rights reserved

Selection Procedure

Algorithm 14.1: ε - Greedy

Step 1: Generate a random number p in the range 0 – 1.

Step 2: If $p \geq \varepsilon$, the algorithm chooses the action with highest reward value.

Step 3: If $p < \varepsilon$, actions are chosen randomly.

Step 4: Update the value of action and repeat for all actions.

With probability ε, the action a is chosen randomly, but rest of the time with probability $1 - \varepsilon$, the best actions based on greedy approach are selected. Thus, there is a balance of exploration as well as exploitation. For example, if ε is 0.2, then 80% best actions are chosen and 20% random actions are chosen.

© Oxford University Press 2021. All rights reserved

RL Agent Types

An agent can be characterized using these methods:

1. Value-based approaches
2. Policy-based approaches
3. Hybrid methods
4. Model-based methods
5. Model free methods

© Oxford University Press 2021. All rights reserved

RL Algorithm Selection

Table 14.3: Algorithm Selection

Model	Environment	On/off	Algorithms Used
Present	Static	Off	Dynamic Programming
Absent	Dynamic	On	SARSA
Absent	Dynamic	-	Temporal Learning, Monte Carlo Methods
Absent	Dynamic	Off	Q-Learning

© Oxford University Press 2021. All rights reserved

Model-Based Learning

Value Iteration Value iteration estimates v_* in term of steps as:

$$v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_*$$

The algorithm for value iteration is given as follows:

Algorithm 14.2: Value Iteration

Step 1: At each iteration $k+1$, for all states $s \in S$.

Step 2: Update $v_{k+1}(s)$ from $v_k(s')$ where s' is the successor state of s.

Step 3: $$v_{k+1}(s) = \max_{a \in A} \sum_{s'} P^a_{ss'} \left[R^a_{ss'} + \gamma v_k(s') \right] \quad (14.15)$$

Slowly, this converges to $v_*(s)$ and finally the policy is:

$$\pi(s) = \arg\max_a \sum_{s'} P^a_{ss'} \left[R^a_{ss'} + \gamma v_k(s') \right] \quad (14.16)$$

© Oxford University Press 2021. All rights reserved

Model-Based Learning

Policy Iteration The second algorithm is policy iteration. The task of finding an optimal policy consists of two steps:

- Policy Evaluation
- Policy Improvement

© Oxford University Press 2021. All rights reserved

Model-Based Learning

Algorithm 14.3: Policy Evaluation

While (True)

Initialize $v(s) = 0$ for all states s

$\Delta = 0$

For each state, $s \in S$ do

$v_{old}(s) = v(s)$

Compute new $v_{new}(s)$ function:

$$v_{new}(s) = \sum_{a} \pi(s, a) \sum_{s'} P_{ss'}^{\pi(s)} + \left[R_{ss'}^{\pi(s)} + \gamma v'(s) \right] \quad (14.17)$$

Repeat until $\max(\Delta, |v_{new}(s) - v_{old}(s)|) < \varepsilon$ then break

Return

Finally, the policy converges to v_*.

© Oxford University Press 2021. All rights reserved

Model-Based Learning

Policy Improvement The policy improvement can be done as follows:

1. Evaluate the policy π with policy evaluation.
2. Solve the Bellman equations for the given policy to get v_π.
3. Improve the policy by applying the greedy approach for policy v_π so that the policy becomes π'.
4. Repeat the process till v_π' policy converges to optimal policy v_*.

The algorithm is given as follows:

Algorithm 14.4: Policy Improvement

For each $s \in S$

Old $= \pi(s)$

$$\pi'(s) = \arg\max_a \sum_{s'} P^a_{ss'} + [R^a_{ss'} + \gamma v(s')] \quad (14.18)$$

If old $\neq \pi(s)$, then GOTO policy evaluation else EXIT.

© Oxford University Press 2021. All rights reserved

Monte-Carlo Methods

MC makes the following assumptions:

1. All episodes should terminate. No matter from where we start, the episode should end.
2. MC methods use value-action functions. These values are computed only after the completion of the episode. So, MC methods are incremental methods.

Algorithm 14.5: MC

Step 1: For a policy π, select state s.
Step 2: Generate an episode.
Step 3: Compute average returns for every first occurrence of state s in an episode.
Step 4: Update the returns and start a new episode. Repeat for all episodes.

© Oxford University Press 2021. All rights reserved

Temporal Difference Learning

Table 14.4: Differences Between Monte-Carlo Method and TD Learning Method

Monte-Carlo Method	Temporal Difference Method
It is easy to understand and does not exploit Markov principle	Exploits Markov principle
This requires termination and episode	Does not require termination

© Oxford University Press 2021. All rights reserved

Temporal Difference Learning

Algorithm 14.6: TD-Learning

1. Initialize policy π.
2. Set $v(s) = 0$ for all states $s \in S$.
3. Repeat for each episode.
4. Initialize s.
5. Repeat for each step of episode.
6. Choose action 'a' and execute it. Observe the new state and new reward.
7. Update value using Bellman Eq. (14.22).
8. Update as s'.
9. Until s is terminal state.

© Oxford University Press 2021. All rights reserved

Q-Learning

Q-Learning is an off-policy method because Q is updated based on policies that are implicit to Q and is better guaranteed for maximum returns. Q indicates Quality. What is Q-value? $Q(s, a)$ is a numerical value assigned to a state-action pair. It means a value of the action that is performed in state 's'.

© Oxford University Press 2021. All rights reserved

Necessary Information

1. Starting state
2. Action
3. Reward
4. New state

Initially, a table called *Q*-table is constructed and filled with initial random values. *Q*-learning involves two policies - learning policy and update policy. *Q*-learning is done by methods like greedy, softmax or softmax plus.

© Oxford University Press 2021. All rights reserved

Update Mechanisms

The update is carried out using the Temporal difference learning (TD) and Bellman equation. The Bellman equation that is used to update the value is given as follows:

$$Q_t(s_t, a_t) = Q_t(s_t, a_t) + \alpha TD_t(s_t, a_t) \quad (14.24)$$

© Oxford University Press 2021. All rights reserved

Algorithm

Algorithm 14.7: *Q*-Learning

1. Set $Q(s_t, a) = 0$ where s_t is the terminal node in an episode.
2. For all states s and action a, set Q Value = 0.
3. Repeat.
4. Select s_t randomly.
5. Choose action a_t from Q using ε- greedy.
6. Perform action a_t such that $r(s_t, a_t) > 0$ and reach the next state.
7. Update the action-value function using TD using Eq. (14.24) and Bellman equation:

$$Q_t(s_t, a_t) = Q_{t-1}(s_t, a_t) + \alpha \times TD_t(s_t, a_t) \quad (14.25)$$

Until the terminal state s is reached.

Here, α is the learning rate. It is between 0 and 1. If α is zero, then nothing is updated and there is no learning at all. When setting it to a higher value such as 0.9, the learning happens fast. γ is the discount factor. It is in the range $0 - 1$. It should be less so that the algorithm can converge.

The inference is simple, the best action is the maximum of Q function as follows:

$$a_t = \arg\max_a(Q(s_t, a))$$

© Oxford University Press 2021. All rights reserved

SARSA Learning

BASED ON VARIABLES

SARSA is based on episodic formulation and discounted factor. Its name is derived from the function $Q(s, a, r, s', a')$, that is, 'r' is the reward and $s'(s_{t+1})$ and $a'(a_{t+1})$ are the new states and actions, respectively. It selects a state 's' and actions 'a' are chosen based on the ε-greedy approach.

© Oxford University Press 2021. All rights reserved

SARSA Learning

SARSA is based on estimated optimal action:

$$a^*_{t+1} = \arg\max_a Q(s_{t+1}, a) \tag{14.26}$$

The ideal rule for updating Q:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r + \gamma Q(s_{t+1}, a_{t+1})) - Q(s_t, a_t) \tag{14.27}$$

This can be given in terms of reward as:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})) - Q(s_t, a_t) \tag{14.28}$$

Where, α and γ are learning rate and discount factor, respectively.

© Oxford University Press 2021. All rights reserved

Algorithm

Algorithm 14.8: SARSA

1. For all states and actions, set $Q(s, a) = 0$.
2. Initialize α, ε and γ.

   Initialize the state s_t.

   Choose action a_t based on selection policy ε-greedy strategy and execute it.

   Get a new state s_{t+1}, observe the reward r_{t+1}.

   Choose action a_{t+1} from Q based on ε-Greedy procedure and execute it.

   Update Action-value function of Eq. (14.28).
3. Update the values $s_t = s_{t+1}$, $a_t = a_{t+1}$.
4. If s' is terminal node, then start a new episode else repeat above steps.

© Oxford University Press 2021. All rights reserved

Summary

1. Reinforcement learning is a branch of machine learning. In reinforcement learning, the agent learns by getting a reward from the environment. It aims to maximize the reward by taking suitable action.
2. Environment is the world where all actions take place. The environment is the framework, where the input, output and rewards are specified.
3. The input for reinforced learning is called a state. The output of reinforced learning is action. The environment is described as the state or state variables or simply as state.
4. Policy takes the input states and processes it, and then suggests an action.
5. Reward is a measure of agent performance.
6. When the environment immediately delivers reward after the execution of action, it is called an immediate reward. When the environment gives the reward after the end like the objective of the game, it is called a long-time reward.
7. The accumulation of all rewards collected over an action is called total reward.
8. Discount factor is the sum of the immediate reward and all other subsequent rewards multiplied by a factor of γ.
9. The interactions of an agent with the environment are modelled as Markov Decision Process (MDP).

© Oxford University Press 2021. All rights reserved

Summary

10. Markov assumption holds good for reinforcement learning. The probability of reaching a state s_t and getting the reward r_t depends solely on the preceding states s_{t-1} and r_{t-1}. All other previous states are not important. This is called Markovian assumption.
11. Dynamic programming is used to solve subproblems. It is an off line algorithm as it needs to wait till the end of episode to update values and actions.
12. Model-based environment means that for any state, the next state and action, the probability distribution is known.

© Oxford University Press 2021. All rights reserved

Summary

13. Value functions summarize all possible future scenarios by averaging over specific returns for a given policy π.
14. The active-value function describes expected returns for a state 's', taking action 'a' and following a policy π.
15. An optimal policy is the one that has better cumulative expected reward over other policies.
16. In model free methods, there is no complete knowledge of the environment. This method works only when MDP terminates.
17. TD is called bootstrapping method because the update is based on the existing estimate. In TD method, in every stop, an estimate of the final reward is calculated at each state. Then, immediately the state-action is updated for every step.
18. Q-Learning is known as an off-policy method because Q is updated based on policies that are implicit to Q and is better guaranteed for maximum returns.
19. SARSA converges to optimal policy and action-value function provided all state-action pairs are visited infinitely often.

© Oxford University Press 2021. All rights reserved